

A hybrid method of feature selection for microarray gene expression data

Cheng-San Yang
Institute of Biomedical Engineering
National Cheng-Kung University
Tainan, Taiwan
p8896117@mail.ncku.edu.tw

Li-Yeh Chuang
Dep. of Chemical Engineering
I-Shou University
Kaohsiung, Taiwan
chuang@isu.edu.tw

Chao-Hsuan Ke, Cheng-Hong Yang
Dep. of Electronic Engineering
National Kaohsiung University of Applied Sciences
Kaohsiung, Taiwan
a092205237, chyang@cc.kuas.edu.tw

Abstract

Gene expression profiles, which represent the state of a cell at a molecular level, have great potential as a medical diagnosis tool. Compared to the number of genes involved, available training data sets generally have a fairly small sample size in cancer type classification. These training data limitations constitute a challenge to certain classification methodologies. A reliable selection method for genes relevant for sample classification is needed in order to speed up the processing rate, decrease the predictive error rate, and to avoid incomprehensibility due to the large number of genes investigated. In this study, we combined information gain and an improved binary particle swarm optimization as a hybrid method to implement feature selection, and the K-nearest neighbor (K-NN) method serves as an evaluator for gene expression data classification problems. Experimental results show that this method effectively simplifies feature selection and reduces the total number of features needed. The classification accuracy obtained by the proposed method has the highest classification accuracy in 6 gene expression data test problems, and is comparative to the classification accuracy, as compared to the best results previously published.

1 Introduction

Microarray gene expression technology has opened the possibility of investigating the activity of thousands of genes simultaneously. Gene expression profiles show the measurement of relative abundance of mRNA corresponding to the genes. Thus, discriminant analysis of microarray data has great potential as a medical diagnosis tool. The goal of microarray data classification is to build an efficient and effective model that identifies the differentially expressed genes and may be used to predict class membership for any unknown samples. The challenges posed in microarray classification are the limited size of samples in comparison to the high dimensionality of the sample, along with experimental variations in measured gene expression levels. The

classification of gene expression data samples involves feature selection and classifier design. Generally, only a small number of genes show a strong correlation with a certain phenotype compared to the total number of genes investigated. Thus, in order to analyze gene expression profiles correctly, feature (gene) selection is most crucial for the classification process. The goal of feature selection is to identify the subset of differentially expressed genes that are potentially relevant for distinguishing the sample classes. A good method of selecting genes relevant for sample classification is needed in order to accelerate the processing rate, decrease the predictive error rate, and to avoid incomprehensibility due to spurious data correlations; it should be based on the number of genes investigated.

Methods used for data reduction, or more specifically for feature selection in the context of microarray data analysis, can be classified into two major groups: filter and wrapper model approaches [5]. In the filter model approach a filtering process precedes the actual the classification process. For each feature a weight value is calculated, and features with better weight values are chosen to represent the original data set. However, the filter approach does not account for interactions between features. The wrapper model approach depends on feature addition or deletion to compose subset features, and uses evaluation function with a learning algorithm to estimate the subset features. This kind of approach is similar to an optimal algorithm that searches for optimal results in a dimension space. The wrapper approach usually conducts a subset search with the optimal algorithm, and then a classification algorithm is used to evaluate the subset.

Particle swarm optimization (PSO) is a population-based stochastic optimization technique, which was developed by Kennedy and Eberhart in 1995 [6]. PSO simulates the social behavior of organisms, such as birds in a flock or fish in a school, and can be described as an automatically evolving system. In PSO, each single candidate solution can be considered "an individual bird in the flock", that is, a particle in

the search space. Each particle makes use of its own memory and knowledge gained by the swarm as a whole to find the best (optimal) solution. In 1997, Kennedy and Eberhart introduced a binary version of PSO (BPSO) [7] to solve discrete problems. In BPSO, each particle represents its position by either the binary value {0} or {1}, and the velocity is treated as a probability change of the particle position. However, BPSO has the same disadvantage as other evolutionary algorithms. After several generations, these algorithms tend to easy get trapped in a local optimum, which might prevent them from converging towards a global optimal solution. In order to circumvent the premature convergence at a local optimum, we incorporated a simple reset operation to create a new *gBest* position. This new *gBest* replaced the original *gBest*, so that all particles were able to leave the local optimal.

In this study, we compared the feature selection performance of the filter and wrapper models, and hybrid the two models to create a new hybrid model for feature selection. To evaluate and compare the proposed method to other feature selection methods, we used the K-nearest neighbor (KNN) algorithm to evaluate the selected features, and to establish the influence on classification accuracy. The results indicate that in terms of the number of genes that need to be selected and classification accuracy the proposed method is superior to other methods in the literature. This paper is organized as follows: a brief overview introducing the methods is presented in Section 2. The proposed method will be explained in Section 3. The experimental framework and settings are described in Section 4. Section 5 consists of the results and a theoretical discussion thereof. Finally, the concluding remarks are offered in Section 6.

2 Related works

2.1 Information Gain

Quinlan [1] proposed a classification algorithm called ID3, which introduces the concept of information gain. Information gain is a measure based method, which is usually used to select best split attributes in decision tree classifiers. The measure indicates to what extent the entire data's entropy is reduced, and identifies the value of each specific attribute. Each feature basis obtains an information gain value, the amount of which is used to decide whether the feature is selected or deleted. Therefore a threshold value for selecting a feature must first established; a feature is selected when the information gain value of this feature is bigger than the threshold value. The formula used to calculate IG is:

$$IG(A) = Info(S) - Info_A(S) \quad (1)$$

where $IG(A)$ is the information gain of the label or attribute A , $Info(S)$ is the entropy of system and $Info_A(S)$ is the system's relative entropy when the value of the label A is known. Let S be the set of n instances and let C be the set of k classes. Let $P(C_i, S)$ be the fraction of the examples in S that have class C_i . Then, the expected information from this class membership is as follows:

$$Info(S) = - \sum_{i=1}^k P(C_i, S) \times \log(P(C_i, S)) \quad (2)$$

If a particular attribute A has v distinct values, the expected information required for the decision tree with A as the root is then the weighted sum of expected information of the subsets of A according to distinct values. Let S_i be the set of instances whose value of attribute A is A_i .

$$Info_A(S) = - \sum_{i=1}^v \frac{|S_i|}{|S|} \times Info(S_i) \quad (3)$$

The higher the information gain, the higher are the chances of getting pure classes in a target class if split on the variable with the highest gain.

2.2 Binary particle swarm optimization

The PSO system is initialized with a population of random solutions. This population searches for an optimal solution by updating generations. In PSO, a potential solution is called a particle. Each particle makes use of its own memory and knowledge gained by the swarm as a whole to find the best (optimal) solution in a d -dimensional search space. The particles have a positional value and velocities which direct their movement.

Each particle is represented by $x_i = (x_{i1}, x_{i2}, \dots, x_{id})$, where d is the dimension numbers. The rate of velocity for the i^{th} particle is represented by $v_i = (v_{i1}, v_{i2}, \dots, v_{id})$ and limited by V_{max} , which is determined by the user. The best previously encountered position of the i^{th} particle (the position with the highest fitness value) is called $pBest_i$ and represented by $p_i = (p_{i1}, p_{i2}, \dots, p_{id})$. The global best value of the entire population is called *gBest* and represented by $g = (g_1, g_2, \dots, g_d)$. At each interaction, the particles are updated according to the following equations:

$$v_{id}^{new} = w \times v_{id}^{old} + c_1 \times rand_1 \times (pbest_{id} - x_i^{old}) + c_2 \times rand_2 \times (gbest_d - x_i^{old}) \quad (4)$$

$$x_{id}^{new} = x_{id}^{old} + v_{id}^{new} \quad (5)$$

where w is the inertia weight, c_1 and c_2 are acceleration (learning) factors, $rand_1$ and $rand_2$ are random numbers. Velocities v_{id}^{new} and v_{id}^{old} are those of the new and old particle, respectively, x_{id}^{old} is the current particle position (solution), and x_{id}^{new} is the updated particle position.

Although PSO was originally introduced as an optimization technique for real-number optimization problems, many optimization problems are set in a space featuring discrete or qualitative distinctions between variables. PSO and BPSO each have their own characteristics. In the first, each particle is composed of a binary variable. Each particle will decide on “yes” or “no”, “true” or “false” or {1} or {0} in this model. In the latter however, the velocity is transformed into a change of probability, namely the probability of the binary variable taking the value {1}. However, the velocity must be restricted to the range [0.0, 1.0]. In order to map the real value number of the velocity to the range, the sigmoid function is proposed to handle the probability of the variables [7].

$$S(v_{pd}^{new}) = \frac{1}{1 + e^{-v_{pd}^{new}}} \quad (6)$$

where $S(v_{pd}^{new})$ denotes the probability of bit x_{pd}^{new} , if $(rand() < S(v_{pd}^{new}))$ then $x_{pd}^{new} = 1$; else $x_{pd}^{new} = 0$, the $rand()$ is a random number selected from a uniform distribution in [0.0, 1.0]. To avoid $S(v_{pd}^{new})$ approaching 0 or 1, a constant V_{max} is used to limit the v_{pd}^{new} , the range of maximum velocity is $[-V_{max}, +V_{max}]$.

2.3 K-nearest neighbor

The K-nearest neighbor (K-NN) method was first introduced by Fix and Hodges in 1951 [2], and is one of the most popular nonparametric methods. The purpose of the algorithm is to classify a new object based on attributes and training samples. The K-nearest neighbor method consists of a supervised learning algorithm where the result of a new instance query is classified based on the majority of the K-nearest neighbor categories. The classifiers do not use any model for fitting and are only based on memory, which works based on a minimum distance from the query instance to the training samples to determine the K-nearest neighbors. Any tied results are solved by a random procedure.

The K-NN method has been successfully applied in many areas: statistical estimation, pattern recognition, artificial intelligence, categorical problems, and feature selection. One advantage of the K-NN method is that it is simple and easy to

implement. Invariant to noisy training data, K-NN is not negatively affected when the training data is large. Disadvantages of the K-NN method are the need to determine parameter K (number of nearest neighbors), calculate the distances between the query instance and all the training samples, sort the distances and determine the nearest neighbors based on the K_{th} minimum distance, as well as determine the categories of the nearest neighbors. Computation cost is quite high because distances from each query instance to all training samples need to be computed. Some indexing (e.g. K-D tree) may reduce this computational cost. In this study, the feature subset was measured by the leave one-out cross-validation (LOOCV) of one nearest neighbor (1-NN). Neighbors are calculated using their Euclidean distance.

The fitness value for the 1-NN evolves according to the LOOCV method for all datasets. In the LOOCV method, a single observation from the original sample is selected as the validation data, and the remaining observations are selected as training data. This is repeated so that each observation in the sample is used once as the validation data. This is essentially the same as K-fold cross-validation where K is equal to the number of observations in the original sample. The obtained classification accuracy is an adaptive functional value.

3 A hybrid method of feature selection

In PSO, each particle adjusts its position according to two fitness value, $pBest$ and $gBest$, to avoid being trapped in a local optimum by fine-tuning the inertia weight. $pBest$ is a local fitness value, whereas $gBest$ constitutes a global fitness value. However, if the $gBest$ value is itself trapped in a local optimum, a search of each particle limit in the same area will occur, thereby preventing superior results of classification. Thus, we propose a method that retires $gBest$ under such circumstances and uses an improved binary particle swarm optimization (IBPSO). By resetting $gBest$ we can avoid BPSO getting trapped in a local optimum, and superior classification result can be achieved with a reduced number of selected genes.

Fig. 1a shows that almost all particles converged near $gBest$ after a certain period. If the $gBest$ value does not change after three iterations, it can be considered stuck at a local optimum. Under such circumstances, the current $gBest$ fitness value (classification accuracy and selected features) is reset to zero, i.e. retired (Fig. 1b). This form of IBPSO skips the local optimum, and searches for superior classification results in an area with a lower number of genes. Fig. 1c shows that the individual particles

will converge towards the reset $gBest$ value and thus leave the local optimum by searching for the new $gBest$ value in a region with a lower number of genes (Fig. 1d). This process achieves superior classification and effectively reduces the number of genes that need to be selected.

In this paper, an improved form of binary PSO (IBPSO) was used since the position of each individual particle can be given in binary form (0 or 1), which adequately reflects the straightforward “yes/no” choice of whether a feature needs to be selected or not. The changes in particle velocity can be interpreted as a change in the probability of finding the particle in one state or another. Since this change is a probability it is limited to a range of $\{0.0 - 1.0\}$.

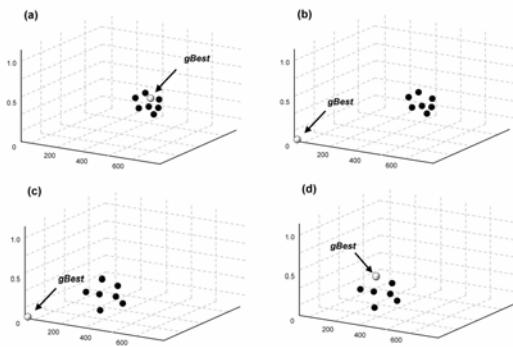


Figure 1. (a) $gBest$ is trapped in a local. (b) $gBest$ is reset to zero. (c) Particle movement after resetting of $gBest$. (d) Particles congregated towards the updated $gBest$ value, improving the individual position. The X-axis is feature numbers, and Y-axis is classification accuracy.

In this study, we hybrid the filter and wrapper model method to select feature genes in microarrays, and used classification algorithms to evaluate the performance of the proposed method. Figure 2 depicts the process of the hybrid filter and wrapper model feature selection method. The filter model part uses information gain (IG) to evaluate the ability of each feature which differentiates between different categories. The reasoning behind this method is that it can calculate the importance of each feature with respect to the class. We used Weka [8] to determine the information value of each feature and sort the features in accordance with their information gain value. Higher values indicate higher discrimination of this feature from other categories, and mean that this feature can be used to calculate classification results effectively. After calculating the information gain values of all features, we implemented a threshold for the results. Since after calculation most information

gain values were zero, not many features have an influence on the categorization of a data set. The threshold in our study is 0 for most data sets. If the information gain values of the features are higher than the threshold, we select the feature, if not, the feature is not selected.

For example, let a microarray data set have 10 gene numbers (10 feature numbers which can be represented by $f_1 f_2 f_3 f_4 f_5 f_6 f_7 f_8 f_9 f_{10}$). If only 5 genes (f_1, f_2, f_4, f_7 and f_{10}) conform to the information gain threshold, only these 5 genes ($f_1 f_2 f_4 f_7 f_{10}$) are used after the wrapper procedure to implement the selection process.

However, when using the filter model selection, the feature number could be reduced dramatically. In order to more effectively remove unwanted features, we used traditional BPSO and the improved BPSO method for wrapper model selection after the initial filter model selection was implemented to select features again, and used KNN algorithm to measure the classification performance.

In BPSO, we used two bits, $\{0\}$ and $\{1\}$, to represent the feature condition. If the number of each particle is identical to the input data dimension number, the i^{th} particle is $p_i = 10011$ in our example, meaning that the f_1, f_7 and f_{10} feature are selected, and the others are not select.

4. Experiment framework

4.1 Data Set

We used 6 multi-category cancer-related human gene expression data sets [3], which were downloaded from <http://www.gems-system.org> to evaluate the performance of the proposed method. The data format is shown in Table 1; it includes the data set name, the number of samples, categories and samples. In order to avoid bias, we implemented a linear scale for each gene expression data value to the range $[0, 1]$.

4.2 Classifier performance

To evaluate the performance of the proposed method, the selected feature subsets were evaluated by leave-one-out cross-validation (LOOCV) of one nearest neighbor (1-NN).

For LOOCV, neighbors are calculated using their Euclidean distance. The fitness value for the 1-NN evolves according to the LOOCV method for all data sets. In the LOOCV method, a single observation from the original sample is selected as the validation data, and the remaining observations are selected as training data. This is repeated so that each observation in the sample is used once as the validation data.

4.3 Experimental parameter set

Principal algorithm parameter settings used were the following. The number of particles used was 30. The two factors $rand_1$ and $rand_2$ are random numbers between $[0, 1]$, whereas c_1 and c_2 are acceleration factors, here $c_1 = c_2 = 2$. The inertia weight w was 1.0. The maximum number of iterations was 100.

5. Result and discussion

In this study, we tested and compared a hybrid filter and wrapper feature selection method's performance on the classification of six multi-category cancer microarray expression data sets. We also proposed an improved BPSO method and compared it to other evolutionary algorithms. After feature selection, the selected feature subsets were evaluated using K-NN classification algorithms.

Table 2 and Table 3 show the accuracies achieved and selected feature number by the filter, wrapper and hybrid model feature selection methods individually. The experimental results show that the accuracy of microarray data which had feature selection implemented was better than without feature selection. For all the feature selection methods, the average of the wrapper model accuracy was better than for the filter model, but the number of selected feature was also higher for the wrapper model than for the filter model. The wrapper model differs from the filter

model in that it is dependent on a classifier and evaluates the combination of feature subsets. The wrapper model can identify interaction amongst all features simultaneously.

However, how many gene subsets are truly necessary to identify cancer categories is still a question under debate [4]. Only filter selection does not reduce the number of features very much; hence another method is needed to reduce the number of features further. In order to select more effective feature subsets, we acceded to a wrapper model method after the implementing the filter approach. Table 2 and Table 3 show that the proposed method effectively increases classification accuracy and selects a smaller number of feature subsets.

During the wrapper phase of the proposed method, we implemented an improved BPSO algorithm, which uses to change original $gBest$ position to prevent the standard BPSO's premature convergence on a local optimal, and compared it to standard BPSO. The experiment showed that the combination of IBPSO and a wrapper model alone or when yet hybrid with a filter method achieves a better performance than standard BPSO. The reason for this is simple: all evolutionary algorithms are prone to converge at a local optimum location. In BPSO, each particle is guided by $pBest$ and $gBest$. However, if $gBest$ is not continuously changed, all of the particles will close in on $gBest$ after several generations, again resulting in a weakened search capability.

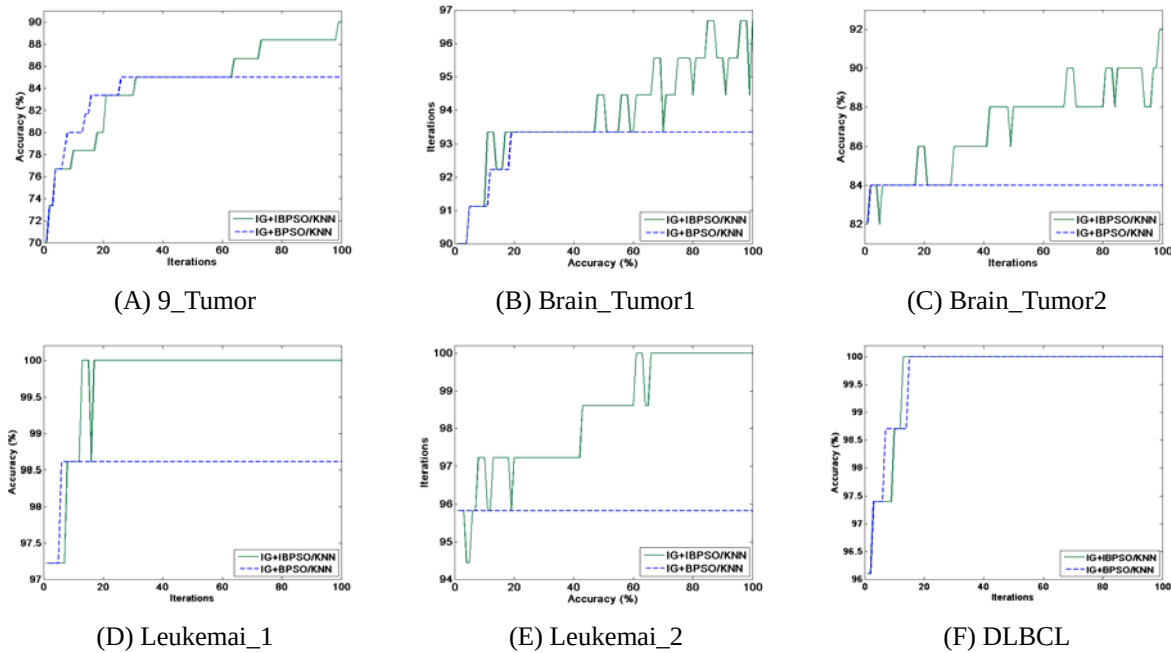


Figure 2 Comparison of distributions of the accuracy of feature selection by IG+BPSO and IG+IBPSO

Figure 2 shows the graphs of the two evaluation algorithm. From the graph we could clear know, after several generations, the BPSO (blue dotted line) could not obtain better performance, that because the position of *gBest* has not changed and lead to many particles cluster around *gBest* instead of exploring the rest of the search space. In BPSO, the velocity is treated as the probability of a bit change of the particle. Close proximity of a particle to *gBest* reduces the probability of this bit occurring.

As mentioned above, each particle will congregate toward *gBest* after several generations. Conversely, if the *gBest* is moved frequently, the particles will not cluster around a certain location. We used this principle to prevent standard BPSO from falling into a local optimum. However, if the position of *gBest* has to be changed, the new position has to be better than the original one. In this study, we employed a simple operation to create a new position for *gBest*, and replace original *gBest*. It could improve the disadvantage of original BPSO which trap into local optimal easily. After the experiment, we proved our proposed could enhance the performance of standard BPSO.

6. Conclusion

In this paper, we hybrid the filter and wrapper model method for microarray classification to implement a feature selection process, and then used KNN to evaluate the classification performance. We used a common filter method to select features firstly and proposed an improved binary particle swarm optimization (IBPSO) to select again. Experimental results show that our method effectively simplified gene (feature) selection and reduced the total number of genes (features) needed. The classification accuracy obtained by the proposed method was the highest of the six gene expression data test problems, and is comparative to the classification accuracy of the other two test problems. The average classification accuracy for the proposed method was increased by 2.85% compared to the best results of the previously published. The proposed method can serve as an ideal pre-processing tool to help optimize the feature selection process, since it increases the classification accuracy and, at the same time, keeps computational resources needed to a minimum. It could also be applied to problems in other areas in the future.

Acknowledgements

This work is partly supported by the National Science Council in Taiwan under grant NSC96-2622-E-151-019-CC3, NSC96-2622-E214-004-CC3, and NSC95-2622-E-214-004.

Reference

- [1] J.R. Quinlan, *Induction of Decision Trees Machine Learning* (1) (1986), pp. 81–106.
- [2] Fix, E., Hodges, J.L., 1951. Discriminatory Analysis—Nonparametric Discrimination: Consistency Properties. Technical Report 21-49-004, Report No. 4, *US Air Force School of Aviation Medicine*, Randolph Field, pp. 261–279.
- [3] A. Statnikov, C.F. Aliferis, I. Tsamardinos, D. Hardin, S. Levy, “A comprehensive evaluation of multicategory classification methods for microarray gene expression cancer diagnosis,” *Bioinformatics*, Vol. 21, No. 5, 2005, pp 631–643.
- [4] T. Li, C. Zhang, and M. Ogihara, “A Comparative Study of Feature Selection and Multiclass Classification Methods for Tissue Classification Based on Gene Expression,” *Bioinformatics*, Vol. 20, No. 15, 2004, pp. 2429–2437.
- [5] Y. Saeys, I. Inza, and P. Larrañaga, “A review of feature selection techniques in bioinformatics,” *Bioinformatics*, 23(19), 2007, pp. 2507–2517.
- [6] J. Kennedy and R.C. Eberhart, Particle swarm optimization. in proceedings of the 1995 IEEE International Conference on Neural Networks, vol. 4, 1995, pp. 1942–1948.
- [7] J. Kennedy and R.C. Eberhart, A discrete binary version of the particle swarm algorithm, Proceedings of the world multiconference on systemics, cybernetics and informatics, *Piscataway*, 1997, pp. 4104–4109.
- [8] E. Frank, M. Hall, L. Trigg, G. Holmes, I.H. Witten, “Data mining in bioinformatics using Weka,” *Bioinformatics*, Vol. 20, No. 15, 2004, pp 2479–2481.

Table 1. Cancer-related human gene expression datasets

Dataset Name	Diagnostic task	Number of		
		Sample	Genes	Classes
9_Tumors	Nine various human Tumor types	60	5726	9
Brain_Tumor1	Five human brain tumor types	90	5920	5
Brain_Tumor2	Four malignant glioma types	50	10367	4
Leukemia1	Acute myelogenous leukemia (AML), acute lymphoblastic leukemia (ALL) B-cell, and ALL T-cell	72	5327	3
Leukemia2	AML, ALL, and mixed-lineage leukemia (MLL)	72	11225	3
DLBCL	Diffuse large B-cell lymphomas and follicular lymphomas	77	5469	2

Table 2. KNN accuracy performance for the six microarray data sets for the filter, wrapper and hybrid filter/wrapper feature selection method

Data set	KNN (Statnikov <i>et al.</i>)[1]	filter	wrapper		Hybrid filter and wrapper	
		IG	BPSO	IBPSO	IG + BPSO	IG + IBPSO
9_Tumors	43.90	66.67	60.00	70.00	85.00	90.00
Brain_Tumor1	87.94	88.89	91.11	93.33	93.33	96.67
Brain_Tumor2	68.67	78.00	80.00	86.00	84.00	92.00
Leukemia1	83.57	93.06	94.44	97.22	98.61	100.0
Leukemia2	87.14	91.67	91.67	97.22	95.83	100.0
DLBCL	86.96	93.51	90.91	96.10	100.0	100.0
Average	76.36	85.30	84.69	89.98	92.80	96.44

Table 3. Feature number for the six microarray data sets for the filter, wrapper and hybrid filter/wrapper feature selection method

Data set	KNN (Statnikov <i>et al.</i>)[1]	filter	wrapper		Hybrid filter and wrapper	
		IG	BPSO	IBPSO	IG + BPSO	IG + IBPSO
9_Tumors	43.90	165	1531	552	49	31
Brain_Tumor1	87.94	1612	2602	435	474	115
Brain_Tumor2	68.67	4465	4158	1081	1855	327
Leukemia1	83.57	848	1811	462	186	34
Leukemia2	87.14	4596	4310	829	459	220
DLBCL	86.96	882	2123	485	252	33